

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»**

ОТЧЕТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ №6

«Работа с БД в СУБД MongoDB»

по дисциплине «Проектирование и реализация баз данных»

Обучающийся Тихонов Роман Александрович

Факультет Прикладной информатики

Группа K3239

Направление подготовки 09.03.03 Прикладная информатика

Образовательная программа Мобильные и сетевые технологии

Преподаватель Говорова Марина Михайловна

Санкт-Петербург

2025/2026

Цель работы:

Овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Выполнение:

1. Практическое задание Лабораторной работы 6.1

1. Установите MongoDB для обеих типов систем (32/64 бита).
2. Проверьте работоспособность системы запуском клиента mongo.
3. Выполните методы:
 - a) db.help()
 - b) db.help
 - c) db.stats()
4. Создайте БД learn.
5. Получите список доступных БД.
6. Создайте коллекцию unicorns, вставив в нее документ {name: 'Aurora', gender: 'f', weight: 450}.
7. Просмотрите список текущих коллекций.
8. Переименуйте коллекцию unicorns.
9. Просмотрите статистику коллекции.
10. Удалите коллекцию.
11. Удалите БД learn.

Команды:

```
db.help()
db.help
db.stats()

use learn
show dbs

db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
show dbs
show collections
db.unicorns.find()

db.unicorns.renameCollection("unicorns2")
```

```
show collections

db.unicorns2.stats()

db.unicorns2.drop()
show collections

db.dropDatabase()
show dbs
```

Результаты выполнения:

```
test> db.stats()
{
  db: 'test',
  collections: Long('0'),
  views: Long('0'),
  objects: Long('0'),
  avgObjSize: 0,
  dataSize: 0,
  storageSize: 0,
  indexes: Long('0'),
  indexSize: 0,
  totalSize: 0,
  scaleFactor: Long('1'),
  fsUsedSize: 0,
  fsTotalSize: 0,
  ok: 1
}
```

```
test> db.help

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in the current database.
  getCollectionInfos Returns an array of documents with collection information, i.e. collection name and options, for the current database.
  runCommand          Runs an arbitrary command on the database.
  adminCommand        Runs an arbitrary command against the admin database.
  aggregate           Runs a specified admin/diagnostic pipeline which does not require an explicit collection
```

```
test> db.help()

Database Class:

  getMongo           Returns the current database connection
  getName            Returns the name of the DB
  getCollectionNames Returns an array containing the names of all collections in the current database.
  getCollectionInfos Returns an array of documents with collection information, i.e. collection name and options, for the current database.
  runCommand         Runs an arbitrary command on the database.
  adminCommand       Runs an arbitrary command against the admin database.
  aggregate          Runs a specified admin/diagnostic pipeline which does not require an underlying collection.
  getSiblingDB       Returns another database without modifying the db variable in the shell environment.
  getCollection      Returns a collection or a view object that is functionally equivalent to
```

```
test> use learn
switched to db learn
learn> show dbs
admin    40.00 KiB
config   92.00 KiB
local    72.00 KiB
learn> db.unicorns.insertOne({name: 'Aurora', gender: 'f', weight: 450})
{
  acknowledged: true,
  insertedId: ObjectId('6a160f09f34f896eddcddcf6')
}
learn> show dbs
admin    40.00 KiB
config   92.00 KiB
learn     8.00 KiB
local    72.00 KiB
learn> show collections
unicorns
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a160f09f34f896eddcddcf6'),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
```

```
learn> show collections
unicorns2
learn> db.unicorns2.find()
[
  {
    _id: ObjectId('6a160f09f34f896eddcddcf6'),
    name: 'Aurora',
    gender: 'f',
    weight: 450
  }
]
```

```
learn> db.unicorns2.stats()
{
  ok: 1,
  capped: false,
  wiredTiger: {
    metadata: { formatVersion: 1 },
    creationString: 'access_pattern_hint=none,allocation_size=4KB,app_metadata=(formatVersion=1),assert=(commit_timestamp=none,durable_timestamp=none,cache_resident=false,checksum=on,colgroups=,collator=,columns=,dictionary=0,encryption=(keyid=,name=),exclusive=false,extractor=,format=btree,huffman_timestamp=oldest_timestamp,enabled=false,file_metadata=,metadata_file=,repair=false),internal_item_max=0,internal_key_max=0,internal_key_truncate=true,max=32KB,leaf_value_max=64MB,log=(enabled=true),lsm=(auto_throttle=true,bloom=true,bloom_bit_count=16,bloom_config=,bloom_hash_count=8,bloom_oldest=,bloom_synchronous=0,compression=0,suffix=),merge_max=15,merge_min=0,memory_page_image_max=0,memory_page_max=10m,os_cache_dirty_max=0,os_cache_max=0,prefix_compression=false,repair=90,tiered_storage=(auth_token=,bucket=,bucket_prefix=,cache_directory=,local_retention=300,name=,object_target_size=0),type=file,value_format=u,v',
    type: 'file',
    uri: 'statistics:table:collection-0-3251595226527028667',
    LSM: {
      'bloom filter false positives': 0,
      'bloom filter hits': 0,
      'bloom filter misses': 0,
      'bloom filter pages evicted from cache': 0,
      'bloom filter pages read into cache': 0,
      'bloom filters in the LSM tree': 0,
      'chunks in the LSM tree': 0,
      'highest merge generation in the LSM tree': 0,
      'queries that could have benefited from a Bloom filter that did not exist': 0,
      'sleep for LSM checkpoint throttle': 0,
      'sleep for LSM merge throttle': 0,
      'total size of bloom filters': 0
    }
  },
  autocommit: {
    'retries for readonly operations': 0,
    'retries for update operations': 0
  },
  backup: {
    'total modified incremental blocks with compressed data': 0,
    'total modified incremental blocks without compressed data': 0
  },
  ...
}
```

```
learn> db.unicorns2.drop()
true
learn> show collections
```

```
learn> db.dropDatabase()
{ ok: 1, dropped: 'learn' }
learn> show dbs
admin    40.00 KiB
config  92.00 KiB
local   72.00 KiB
learn>
```

2. Практическая часть 2

2.1.1 Вставка документов в коллекцию

Команды:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600,
gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450,
gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight:
984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575,
gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'],
weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733,
gender: 'f', vampires: 40});
```

```
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690,  
gender: 'm', vampires: 39});  
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421,  
gender: 'm', vampires: 2});  
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight:  
601, gender: 'f', vampires: 33});  
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight:  
650, gender: 'm', vampires: 54});  
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540,  
gender: 'f'}));  
  
document = ({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704,  
gender: 'm', vampires: 165})  
db.unicorns.insert(document)
```

Результат выполнения:

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a16149cf34f896eddcddcdf7'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a1614ecf34f896eddcddcdf8'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a1614f8f34f896eddcddcdf9'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a1614fff34f896eddcddcdfa'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a161505f34f896eddcddcdfb'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a161517f34f896eddcddcdfc'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  .
]
```



```
},
{
  _id: ObjectId('6a16151cf34f896eddcddcdfd'),
  name: 'Kenny',
  loves: [ 'grape', 'lemon' ],
  weight: 690,
  gender: 'm',
  vampires: 39
},
{
  _id: ObjectId('6a161520f34f896eddcddcdfe'),
  name: 'Raleigh',
  loves: [ 'apple', 'sugar' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
{
  _id: ObjectId('6a161526f34f896eddcddcdff'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('6a16152af34f896eddcddce00'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a16152ef34f896eddcddce01'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a16157cf34f896eddcddce02'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
```


2.2.1 Практическое задание

1) Сформируйте запросы для вывода списков самцов и самок единорогов.

Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

Команды:

```
db.unicorns.find({ gender: "m" }).sort({ name: 1 })
db.unicorns.find({ gender: "f" }).sort({ name: 1 }).limit(3)
```

Результат выполнения:

```
learn> db.unicorns.find({ gender: "m" }).sort({ name: 1 })
[
  {
    _id: ObjectId('6a16157cf34f896eddcde02'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a16149cf34f896eddcddf7'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a16152ef34f896eddcde01'),
    _id: ObjectId('6a16151cf34f896eddcddf9'),
    name: 'Kenny', 'e', 'carrot' ],
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a16157cf34f896eddcde02'),
    name: 'Dunx',
    _id: ObjectId('6a16152af34f896eddcde00'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,5
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a161520f34f896eddcddf9'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a1614fff34f896eddcdf9'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a1614f8f34f896eddcddf9'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn>
```

```

learn> db.unicorns.find({ gender: "f" }).sort({ name: 1 }).limit(3)
[
  {
    _id: ObjectId('6a1614ecf34f896eddcddcd8'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a161517f34f896eddcddcd8'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a161526f34f896eddcddcd8'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn>

```

2) Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций findOne и limit.

Команды:

```
db.unicorns.find({ gender: "f", loves: "carrot" }).sort({ name: 1 })
```

Результат выполнения:

```

learn> db.unicorns.find({ gender: "f", loves: "carrot" }).sort({ name: 1 })
[
  {
    _id: ObjectId('6a1614ecf34f896eddcdf8'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a16152ef34f896eddcdf01'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a161505f34f896eddcdfb'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  }
]
learn> db.unicorns.findOne({ gender: "f", loves: "carrot" })
{
  _id: ObjectId('6a1614ecf34f896eddcdf8'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> db.unicorns.find({ gender: "f", loves: "carrot" }).limit(1)
[
  {
    _id: ObjectId('6a1614ecf34f896eddcdf8'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>

```

2.2.2 Практическое задание

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

Команды:

```
db.unicorns.find({ gender: "m" }, { loves: 0, gender: 0, _id: 0 }).sort({ name: 1 })
```

Результат выполнения:

```

learn> db.unicorns.find(
...   { gender: "m" },
...   { loves: 0, gender: 0 }
... ).sort({ name: 1 })
[
  {
    _id: ObjectId('6a16157cf34f896edddcdce02'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  },
  {
    _id: ObjectId('6a16149cf34f896edddcdcdf7'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a16151cf34f896edddcdcdfd'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a16152af34f896edddcdce00'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a161520f34f896edddcdcdfef'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a1614fff34f896edddcdcdfa'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a1614f8f34f896edddcdcdf9'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  }
]
learn>

```

2.2.3 Практическое задание

Вывести список единорогов в обратном порядке добавления.

Команды:

```
db.unicorns.find().sort({ $natural: -1 })
```

Результат выполнения:

```
learn> db.unicorns.find().sort({ $natural: -1 })
[
  {
    _id: ObjectId('6a16157cf34f896eddcddce02'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a16152ef34f896eddcddce01'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a16152af34f896eddcddce00'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a161526f34f896eddcddcdf'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
]
```

2.2.4 Практическое задание

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

Команды:

```
db.unicorns.find({}, { _id: 0, name: 1, loves: { $slice: 1 } })
```

Результат выполнения:

```
learn> db.unicorns.find({}, { _id: 0, name: 1, loves: { $slice: 1 } })
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>
```

2.3.1 Практическое задание

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

Команды:

```
db.unicorns.find({ gender: "f", weight: { $gte: 500, $lte: 700 } }, { _id: 0 })
```

Результат выполнения:

```
learn> db.unicorns.find({ gender: "f", weight: { $gte: 500, $lte: 700 } }, { _id: 0 })
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

2.3.2 Практическое задание

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

Команды:

```
db.unicorns.find({ gender: "m", weight: { $gte: 500 }, loves: { $all: ["grape", "lemon"] } }, { _id: 0 })
```

Результат выполнения:

```
learn> db.unicorns.find({ gender: "m", weight: { $gte: 500 }, loves: { $all: ["grape", "lemon"] } }, { _id: 0 })
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn>
```

2.3.3 Практическое задание

Найти всех единорогов, не имеющих ключ vampires.

Команды:

```
db.unicorns.find({ vampires: { $exists: false } })
```

Результат выполнения:

```
learn> db.unicorns.find({ vampires: { $exists: false } })
[
  {
    _id: ObjectId('6a16152ef34f896eddcde01'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn>
```

2.3.4 Практическое задание

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

Команды:

```
db.unicorns.find({ gender: "m" }, { _id: 0, name: 1, loves: { $slice: 1 } }).sort({ name: 1 })
```

Результат выполнения:


```
learn> db.unicorns.find({ gender: "m" }, { _id: 0, name: 1, loves: { $slice: 1 } }).sort({ name: 1 })
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

3. Практическая часть 3

3.1.1 Практическое задание

1) Создайте коллекцию towns, включающую 3 документа.

Команды:

```
db.towns.insertMany([
  {
    name: "Punxsutawney",
    populatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: [""],
    mayor: { name: "Jim Wehrle" }
  },
  {
    name: "New York",
    populatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  },
  {
    name: "Portland",
    populatiuon: 528000,
    last_sensus: ISODate("2009-07-20"),
    famous_for: ["beer", "food"],
    mayor: { name: "Sam Adams", party: "D" }
  }
])
db.towns.find()
```

Результат выполнения:

```
learn> db.towns.find()
[
  {
    _id: ObjectId('6a170964c3a07d94a2cdcdf6'),
    name: 'Punxsutawney',
    populatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ '' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a170964c3a07d94a2cdcdf7'),
    name: 'New York',
    populatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a170964c3a07d94a2cdcdf8'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
```

2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.

Команды:

```
db.towns.find({ "mayor.party": "I" }, { _id: 0, name: 1, mayor: 1 })
```

Результат выполнения:

```
learn> db.towns.find({ "mayor.party": "I" }, { _id: 0, name: 1, mayor: 1 })
[
  {
    name: 'New York',
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  }
]
learn>
```

3) Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

Команды:

```
db.towns.find({ "mayor.party": { $exists: false } }, { _id: 0, name: 1, mayor: 1 })
```

Результат выполнения:

```
learn> db.towns.find({ "mayor.party": { $exists: false } }, { _id: 0, name: 1, mayor: 1 })
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
```

3.1.2 Практическое задание

1) Сформировать функцию для вывода списка самцов единорогов.

Команды:

```
function showMaleUnicorns() {
  db.unicorns.find({ gender: "m" }).sort({ name: 1 }).forEach(function(u) {
    print(u.name);
  });
}
showMaleUnicorns();
```

Результат выполнения:

```
learn> function showMaleUnicorns() {
...   db.unicorns.find({ gender: "m" }).sort({ name: 1 }).forEach(function(u) {
...     print(u.name);
...   });
... }
[Function: showMaleUnicorns]
learn> showMaleUnicorns();
Dunx
Horny
Kenny
Pilot
Raleigh
Roooooodles
Unicrom
learn>
```

2) Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке. Вывести результат, используя `forEach`.

Команды:

```
var cursor = db.unicorns.find({ gender: "m" }).sort({ name: 1 }).limit(2);
cursor.forEach(function(u) {
  printjson(u);
});
```

Результат выполнения:

```
learn> var cursor = db.unicorns.find({ gender: "m" }).sort({ name: 1 }).limit(2);

learn> cursor.forEach(function(u) {
...   printjson(u);
... });
{
  _id: ObjectId('6a16157cf34f896eddcddce02'),
  name: 'Dunx',
  loves: [
    'grape',
    'watermelon'
  ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
{
  _id: ObjectId('6a16149cf34f896eddcddcf7'),
  name: 'Horny',
  loves: [
    'carrot',
    'papaya'
  ],
  weight: 600,
  gender: 'm',
  vampires: 63
}

learn>
```

3.2.1 Практическое задание

Вывести количество самок единорогов весом от полутонны до 600 кг.

Команды:

```
db.unicorns.find({ gender: "f", weight: { $gte: 500, $lte: 600 } }).count()
```

Результат выполнения:

```
learn> db.unicorns.find({ gender: "f", weight: { $gte: 500, $lte: 600 } }).count()
2
learn> ■
```

3.2.2 Практическое задание

Вывести список предпочтений.

Команды:

```
db.unicorns.distinct("loves")
```

Результат выполнения:

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn>
```

3.2.3 Практическое задание

Посчитать количество особей единорогов обоих полов.

Команды:

```
db.unicorns.aggregate([ { $group: { _id: "$gender", count: { $sum: 1 } } } ])
```

Результат выполнения:

```
learn> db.unicorns.aggregate([ { $group: { _id: "$gender", count: { $sum: 1 } } } ])
```

```
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
```

```
learn> ■
```

3.3.1 Практическое задание

Выполнить команду и проверить содержимое коллекции unicorns.

Команды:

```
db.unicorns.save({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})
db.unicorns.find({ name: "Barney" })
```

Результат выполнения:

```
learn> db.unicorns.find({ name: "Barney" })
[
  {
    _id: ObjectId('6a173692134d9bbfe1cdcdf6'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
learn>
```

3.3.2 Практическое задание

Для самки единорога Ауна внести изменения в БД: теперь ее вес 800, она убила 51 вапмира. Проверить содержимое коллекции unicorns.

Команды:

```
db.unicorns.find({ name: "Ayna" })
db.unicorns.updateOne({ name: "Ayna" }, { $set: { weight: 800, vampires: 51 } })
db.unicorns.find({ name: "Ayna" })
```

Результат выполнения:

```
learn> db.unicorns.find({ name: "Ayna" })
[
  {
    _id: ObjectId('6a173240018a535a32cdce09'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  }
]
learn> db.unicorns.updateOne({ name: "Ayna" }, { $set: { weight: 800, vampires: 51 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: "Ayna" })
[
  {
    _id: ObjectId('6a173240018a535a32cdce09'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn>
```

3.3.3 Практическое задание

Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул. Проверить содержимое коллекции unicorns.

Команды:

```
db.unicorns.find({ name: "Raleigh" })
db.unicorns.updateOne({ name: "Raleigh" }, { $addToSet: { loves: "redbull" } })
db.unicorns.find({ name: "Raleigh" })
```

Результат выполнения:

```

learn> db.unicorns.find({ name: "Raleigh" })
[
  {
    _id: ObjectId('6a173241018a535a32cdce0b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> db.unicorns.updateOne({ name: "Raleigh" }, { $addToSet: { loves: "redbull" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: "Raleigh" })
[
  {
    _id: ObjectId('6a173241018a535a32cdce0b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn> 

```

3.3.4 Практическое задание

Всем самцам единорогов увеличить количество убитых вампиров на 5.

Проверить содержимое коллекции unicorns.

Команды:

```

db.unicorns.find({ gender: "m" }, { _id: 0, name: 1, vampires: 1 }).sort({ name: 1 })
db.unicorns.updateMany({ gender: "m" }, { $inc: { vampires: 5 } })
db.unicorns.find({ gender: "m" }, { _id: 0, name: 1, vampires: 1 }).sort({ name: 1 })

```

Результат выполнения:


```

learn> db.unicorns.find({ gender: "m" }, { _id: 0, name: 1, vampires: 1 }).sort({ name: 1 })
[
  { name: 'Barney' },
  { name: 'Dunx', vampires: 165 },
  { name: 'Horny', vampires: 63 },
  { name: 'Kenny', vampires: 39 },
  { name: 'Pilot', vampires: 54 },
  { name: 'Raleigh', vampires: 2 },
  { name: 'Rooooooodles', vampires: 99 },
  { name: 'Unicrom', vampires: 182 }
]
learn> db.unicorns.updateMany({ gender: "m" }, { $inc: { vampires: 5 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find({ gender: "m" }, { _id: 0, name: 1, vampires: 1 }).sort({ name: 1 })
[
  { name: 'Barney', vampires: 5 },
  { name: 'Dunx', vampires: 170 },
  { name: 'Horny', vampires: 68 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Unicrom', vampires: 187 }
]
learn>

```

3.3.5 Практическое задание

Изменить информацию о городе Портланд: мэр этого города теперь беспартийный. Проверить содержимое коллекции towns.

Команды:

```

db.towns.find({ name: "Portland" })
db.towns.updateOne({ name: "Portland" }, { $unset: { "mayor.party": 1 } })
db.towns.find({ name: "Portland" })

```

Результат выполнения:

```

learn> db.towns.find({ name: "Portland" })
[
  {
    _id: ObjectId('6a170964c3a07d94a2cdcdf8'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns.updateOne({ name: "Portland" }, { $unset: { "mayor.party": 1 } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({ name: "Portland" })
[
  {
    _id: ObjectId('6a170964c3a07d94a2cdcdf8'),
    name: 'Portland',
    populatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn>

```

3.3.6 Практическое задание

Изменить информацию о самце единорога Pilot: теперь он любит и шоколад. Проверить содержимое коллекции unicorns.

Команды:

```

db.unicorns.find({ name: "Pilot" })
db.unicorns.updateOne({ name: "Pilot" }, { $push: { loves: "chocolate" } })
db.unicorns.find({ name: "Pilot" })

```

Результат выполнения:

```

learn> db.unicorns.find({ name: "Pilot" })
[
  {
    _id: ObjectId('6a173241018a535a32cdce0d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> db.unicorns.updateOne({ name: "Pilot" }, { $push: { loves: "chocolate" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: "Pilot" })
[
  {
    _id: ObjectId('6a173241018a535a32cdce0d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn>

```

3.3.7 Практическое задание

Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны. Проверить содержимое коллекции unicorns.

Команды:

```

db.unicorns.find({ name: "Aurora" })
db.unicorns.updateOne({ name: "Aurora" }, { $addToSet: { loves: { $each: ["sugar", "lemon"] } } })
db.unicorns.find({ name: "Aurora" })

```

Результат выполнения:

```

learn> db.unicorns.find({ name: "Aurora" })
[
  {
    _id: ObjectId('6a173240018a535a32cdce05'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> db.unicorns.updateOne({ name: "Aurora" }, { $addToSet: { loves: { $each: ["sugar", "lemon"] } } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ name: "Aurora" })
[
  {
    _id: ObjectId('6a173240018a535a32cdce05'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>

```

3.4.1 Практическое задание

Создайте коллекцию towns, включающую прописанные документы.

Удалите документы с беспартийными мэрами. Проверьте содержание коллекции. Очистите коллекцию. Просмотрите список доступных коллекций.

Команды:

```

db.towns.insertMany([
  {
    name: "Punxsutawney",
    popujatiuon: 6200,
    last_sensus: ISODate("2008-01-31"),
    famous_for: ["phil the groundhog"],
    mayor: { name: "Jim Wehrle" }
  },
  {
    name: "New York",
    popujatiuon: 22200000,
    last_sensus: ISODate("2009-07-31"),
    famous_for: ["status of liberty", "food"],
    mayor: { name: "Michael Bloomberg", party: "I" }
  }
])

```

```
    },  
    {  
      name: "Portland",  
      popujatiuon: 528000,  
      last_sensus: ISODate("2009-07-20"),  
      famous_for: ["beer", "food"],  
      mayor: { name: "Sam Adams", party: "D" }  
    }  
  ])  
  
db.towns.find()  
  
db.towns.deleteMany({ "mayor.party": { $exists: false } })  
db.towns.find()  
  
db.towns.deleteMany({})  
db.towns.find()  
show collections
```

Результат выполнения:

```

learn> db.towns.find()
[
  {
    _id: ObjectId('6a173ecf5d79aabaa6cdcdf6'),
    name: 'Punxsutawney',
    popujatiuon: 6200,
    last_sensus: ISODate('2008-01-31T00:00:00.000Z'),
    famous_for: [ 'phil the groundhog' ],
    mayor: { name: 'Jim Wehrle' }
  },
  {
    _id: ObjectId('6a173ecf5d79aabaa6cdcdf7'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a173ecf5d79aabaa6cdcdf8'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn> db.towns.deleteMany({ "mayor.party": { $exists: false } })
{ acknowledged: true, deletedCount: 1 }
learn> db.towns.find()
[
  {
    _id: ObjectId('6a173ecf5d79aabaa6cdcdf7'),
    name: 'New York',
    popujatiuon: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'status of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a173ecf5d79aabaa6cdcdf8'),
    name: 'Portland',
    popujatiuon: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]

```

4. Практическая часть 4

4.1.1 Практическое задание

1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

Команды:

```
db.zones.drop()

db.zones.insertMany([
  { _id: "gl", name: "Glade", description: "Солнечные поляны, много цветов и фруктов" },
  { _id: "sw", name: "Swamp", description: "Болота и туман: влажно, много лягушек и трав" },
  { _id: "ds", name: "Desert", description: "Пустыня: жарко, мало воды, но много редких трав" },
  { _id: "lk", name: "Lake", description: "Озёрная зона: вода, прохлада, сладкие водоросли" }
])

db.zones.find()
```

Результат выполнения:

```
learn> db.zones.find()
[
  {
    _id: 'gl',
    name: 'Glade',
    description: 'Солнечные поляны, много цветов и фруктов'
  },
  {
    _id: 'sw',
    name: 'Swamp',
    description: 'Болота и туман: влажно, много лягушек и трав'
  },
  {
    _id: 'ds',
    name: 'Desert',
    description: 'Пустыня: жарко, мало воды, но много редких трав'
  },
  {
    _id: 'lk',
    name: 'Lake',
    description: 'Озёрная зона: вода, прохлада, сладкие водоросли'
  }
]
learn>
```

2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.

Команды:

```
db.unicorns.updateOne(
  { name: "Horny" },
  { $set: { zone: { $ref: "zones", $id: "gl" } } }
)
db.unicorns.updateOne(
  { name: "Aurora" },
  { $set: { zone: { $ref: "zones", $id: "lk" } } }
)
db.unicorns.updateOne(
  { name: "Kenny" },
  { $set: { zone: { $ref: "zones", $id: "sw" } } }
)
db.unicorns.updateOne(
  { name: "Pilot" },
  { $set: { zone: { $ref: "zones", $id: "ds" } } }
)

db.unicorns.find({ zone: { $exists: true } }, { _id: 0, name: 1, zone: 1 })
```

Результат выполнения:

```

learn> db.unicorns.updateOne(
...   { name: "Aurora" },
...   { $set: { zone: { $ref: "zones", $id: "lk" } } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>

learn> db.unicorns.updateOne(
...   { name: "Kenny" },
...   { $set: { zone: { $ref: "zones", $id: "sw" } } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn>

learn> db.unicorns.updateOne(
...   { name: "Pilot" },
...   { $set: { zone: { $ref: "zones", $id: "ds" } } }
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({ zone: { $exists: true } }, { _id: 0, name: 1, zone: 1 })
[
  { name: 'Horny', zone: DBRef('zones', 'gl') },
  { name: 'Aurora', zone: DBRef('zones', 'lk') },
  { name: 'Kenny', zone: DBRef('zones', 'sw') },
  { name: 'Pilot', zone: DBRef('zones', 'ds') }
]

```

3) Проверьте содержание коллекции единорогов.

Команды:

```

var u = db.unicorns.findOne({ name: "Horny" })
u.zone

db[u.zone.collection].findOne({ _id: u.zone.oid })

```

Результат выполнения:

```
learn> var u = db.unicorns.findOne({ name: "Horny" })

learn> u.zone
DBRef('zones', 'gl')
learn> db[u.zone.collection].findOne({ _id: u.zone.oid })
{
  _id: 'gl',
  name: 'Glade',
  description: 'Солнечные поляны, много цветов и фруктов'
}
learn> █
```

4.2.1 Практическое задание

Проверьте, можно ли задать для коллекции `unicorns` индекс для ключа `name` с флагом `unique`.

Команды:

```
db.unicorns.createIndex({ name: 1 }, { unique: true })
db.unicorns.getIndexes()
```

Результат выполнения:

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true })
name_1
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> █
```

Вывод по заданию 4.2.1:

Уникальный индекс по полю `name` обеспечивает, что каждое значение `name` в коллекции должно быть неповторяющимся. При попытке добавить документ с уже существующим `name` MongoDB отклонит операцию и выдаст ошибку `E11000 duplicate key error`.

4.3.1 Практическое задание

Получите информацию о всех индексах коллекции `unicorns`. Удалите все индексы, кроме индекса для идентификатора. Попытайтесь удалить индекс для идентификатора.

Команды:

```
db.unicorns.getIndexes()
db.unicorns.dropIndexes()
```

```
db.unicorns.getIndexes()
db.unicorns.dropIndex("_id_")
```

Результат выполнения:

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
learn> db.unicorns.dropIndexes()
{
  nIndexesWas: 2,
  msg: 'non-_id indexes dropped for collection',
  ok: 1
}
learn> db.unicorns.getIndexes()
[ { v: 2, key: { _id: 1 }, name: '_id_' } ]
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
learn> ■
```

Вывод по заданию 4.3.1:

Индекс `_id` создаётся MongoDB автоматически для каждой коллекции и обеспечивает уникальность значений поля `_id` у документов. Этот индекс является системным и необходим для корректной работы коллекции, поэтому MongoDB запрещает его удаление.

4.4.1 Практическое задание

1) Создайте объемную коллекцию `numbers`, задействовав курсор. Выберите последних четыре документа.

Команды:

```
for (i = 0; i < 100000; i++) { db.numbers.insertOne({ value: i }) }
db.numbers.countDocuments()
db.numbers.find().sort({ value: -1 }).limit(4)
```

Результат выполнения:

```

learn> for (i = 0; i < 100000; i++) { db.numbers.insertOne({ value: i }) }

{
  acknowledged: true,
  insertedId: ObjectId('6a1752e2d6ef2bc0c1cf5495')
}
learn>

learn> db.numbers.countDocuments()
100000
learn> db.numbers.find().sort({ value: -1 }).limit(4)
[
  { _id: ObjectId('6a1752e2d6ef2bc0c1cf5495'), value: 99999 },
  { _id: ObjectId('6a1752e2d6ef2bc0c1cf5494'), value: 99998 },
  { _id: ObjectId('6a1752e2d6ef2bc0c1cf5493'), value: 99997 },
  { _id: ObjectId('6a1752e2d6ef2bc0c1cf5492'), value: 99996 }
]

```

2) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра `executionTimeMillis`)

Команды:

```
db.numbers.explain("executionStats").find().sort({ value: -1 }).limit(4)
```

Результат выполнения:

```

learn> db.numbers.explain("executionStats").find().sort({ value: -1 }).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'BA27D965',
    planCacheKey: '7A892B81',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'SORT',
      sortPattern: { value: -1 },
      memLimit: 104857600,
      limitAmount: 4,
      type: 'simple',
      inputStage: { stage: 'COLLSCAN', direction: 'forward' }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 83,
    totalKeysExamined: 0,
    totalDocsExamined: 100000,
    executionStages: {
      isCached: false,
      stage: 'SORT',
      nReturned: 4,
      executionTimeMillisEstimate: 74,
      works: 100006,
      advanced: 4,

```

Вывод:

Результаты explain показывают, что для получения последних 4 документов сервер выполнил полный просмотр коллекции (COLLSCAN) и обработал все 100 000 записей. Такой способ неэффективен, что подтверждается сравнительно большим временем выполнения запроса — 83 мс.

3) Создайте индекс для ключа value. Получите информацию о всех индексах коллекции numbers. Выполните запрос 2. Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса? Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Команды:

```
db.numbers.createIndex({ value: 1 })
db.numbers.getIndexes()
db.numbers.find().sort({ value: -1 }).limit(4)
db.numbers.explain("executionStats").find().sort({ value: -1 }).limit(4)
```

Результат выполнения:

```
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn> db.numbers.find().sort({ value: -1 }).limit(4)
[
  { _id: ObjectId('6a1752e2d6ef2bc0c1cf5495'), value: 99999 },
  { _id: ObjectId('6a1752e2d6ef2bc0c1cf5494'), value: 99998 },
  { _id: ObjectId('6a1752e2d6ef2bc0c1cf5493'), value: 99997 },
  { _id: ObjectId('6a1752e2d6ef2bc0c1cf5492'), value: 99996 }
]
```

```
learn> db.numbers.explain("executionStats").find().sort({ value: -1 }).limit(4)
{
  explainVersion: '1',
  queryPlanner: {
    namespace: 'learn.numbers',
    parsedQuery: {},
    indexFilterSet: false,
    queryHash: 'BA27D965',
    planCacheKey: '7A892B81',
    optimizationTimeMillis: 0,
    maxIndexedOrSolutionsReached: false,
    maxIndexedAndSolutionsReached: false,
    maxScansToExplodeReached: false,
    prunedSimilarIndexes: false,
    winningPlan: {
      isCached: false,
      stage: 'LIMIT',
      limitAmount: 4,
      inputStage: {
        stage: 'FETCH',
        inputStage: {
          stage: 'IXSCAN',
          keyPattern: { value: 1 },
          indexName: 'value_1',
          isMultiKey: false,
          multiKeyPaths: { value: [] },
          isUnique: false,
          isSparse: false,
          isPartial: false,
          indexVersion: 2,
          direction: 'backward',
          indexBounds: { value: [ '[MaxKey, MinKey]' ] }
        }
      }
    },
    rejectedPlans: []
  },
  executionStats: {
    executionSuccess: true,
    nReturned: 4,
    executionTimeMillis: 1,
    totalKeysExamined: 4,
    totalDocsExamined: 4,
  }
}
```


Вывод:

Отчёт explain показывает, что после создания индекса MongoDB начала использовать индексное сканирование (IXSCAN). В результате для получения 4 последних записей системе понадобилось просмотреть лишь 4 ключа индекса и 4 соответствующих документа, вместо обработки всей коллекции. Сравнение результатов наглядно демонстрирует влияние индекса на производительность: без индекса выполнялось полное сканирование 100 000 документов и запрос занимал 83 мс, а с индексом было проверено только 4 документа и время сократилось до 1 мс. Таким образом, эксперимент подтверждает, что индексация критически важна для ускорения запросов на больших объёмах данных, поскольку позволяет избежать полного перебора коллекции и значительно уменьшает время выполнения операций.

Выводы:

В процессе выполнения лабораторных работ № 6.1 и № 6.2 были получены практические навыки работы с документо-ориентированной СУБД MongoDB. Были рассмотрены базовые элементы модели данных — документы и коллекции, а также особенности гибкой структуры хранения, позволяющей размещать в одной коллекции записи с различным набором полей без предварительного задания строгой схемы. Основная часть работы была посвящена освоению CRUD-операций. Были отработаны методы добавления данных (`insert`), чтения и поиска документов (`find`, `findOne`), а также построение запросов с использованием логических операторов (`$gte`, `$all`, `$exists`). Отдельное внимание уделялось выборкам по вложенным полям с применением точечной нотации, управлению выдачей через сортировку (`sort`), ограничение количества результатов (`limit`) и настройку набора возвращаемых полей с помощью проекции, включая извлечение части элементов массивов через оператор `$slice`. Дополнительно были изучены средства агрегирования, позволяющие выполнять анализ данных: подсчёт количества документов, получение уникальных значений (`distinct`) и группировка с вычислениями на основе pipeline-агрегации (`aggregate`, `$group`).

На заключительном этапе были закреплены навыки изменения и удаления данных. Были применены атомарные операторы обновления (`$set`, `$push`, `$inc`, `$unset`), позволяющие корректировать отдельные поля и массивы без полной замены документа, а также рассмотрены массовые операции обновления (`updateMany`) и удаления.

Важным результатом стало изучение индексации и её влияния на производительность. На примере большой коллекции было показано, что создание индекса существенно ускоряет выполнение запросов: вместо полного просмотра коллекции (`COLLSCAN`) система использует

индексное сканирование (IXSCAN), что сокращает количество просматриваемых записей и уменьшает время выполнения запроса с десятков миллисекунд до единиц. В итоге был получен комплексный опыт работы с MongoDB, охватывающий создание, поиск, изменение, удаление и оптимизацию доступа к данным.